A REPORT OF SIX WEEKS INTERNSHIP

at

INDIAN INSTITUTE OF TECHNOLOGY, JAMMU

IIT-JAMMU

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

(Computer Science and Engineering)

JUNE-JULY ,2024

SUBMITTED BY:

NAME : **KARAN SINGH**

UNIVERSITY ROLL NO. : **2203482**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

GURU NANAK DEV ENGINEERING COLLEGE LUDHIANA

(An Autonomous College Under UGC ACT)

CERTIFICATE OF INTERNSHIP-IIT JAMMU

भारतीय प्रौद्योगिकी संस्थान जम्मू
INDIAN INSTITUTE OF TECHNOLOGY JAMMU

Career Development Services

CERTIFICATE
OF COMPLETION

THIS IS PRESENTED TO

Mr. Karan Singh

for successfully completing the RISE UP 2024 Internship Program at the **Indian Institute of Technology Jammu** under the supervision of Dr. Sidharth Maheshwari

for the period of 06 weeks starting from 24 / 06 / 2024.

DR. VINIT JAKHETIYA
PROFESSOR IN-CHARGE
CAREER DEVELOPMENT SERVICES

Indian Institute of Technology Jammu,
Jagti, NH-44, Jammu - 181221, J&K, India
P: +91-191-2570289
E: riseup@iitjammu.ac.in
W: www.iitjammu.ac.in

CANDIDATE'S DECLARATION

I, **KARAN SINGH**, hereby declare that I have successfully completed a six-week internship at **Indian Institute of Technology, Jammu (IIT Jammu)** from **24th June to 31st July**, as part of the partial fulfillment of the requirements for the award of the degree of B.Tech in Computer Science and Engineering at Guru Nanak Dev Engineering College, Ludhiana.

The work presented in this internship report is an authentic and genuine account of the project undertaken and is being submitted to the Department of Computer Science and Engineering at Guru Nanak Dev Engineering College, Ludhiana.

Karan Singh

Signature of the Student

ABSTRACT

This report summarizes the work completed during a six-week internship at **IIT Jammu** from **24th June to 31st July**, focused on the project **"TinyML Applications for the Edge"**. The project involved using the **Arduino TinyML Kit** to implement machine learning models on low-power microcontrollers, with a specific focus on generating and processing sine wave data using the **Arduino Nano 33 BLE Sense** and **DAC module (DFR0972)**. The work explored I2C communication, data transfer, and the application of TinyML in edge computing devices. The project's outcomes demonstrate the potential for optimizing machine learning on resource-constrained devices for real-world applications.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Indian Institute of Technology, Jammu (IIT Jammu)** for providing me the opportunity to undertake this enriching internship. I am deeply grateful to **Dr. Sidharth Maheshwari**, my internship supervisor, for their invaluable guidance, insightful feedback, and constant support throughout the project.

I would also like to extend my thanks to the entire **Department of Computer Science and Engineering at IIT Jammu**, whose resources and expertise helped shape the direction of my project. The collaborative environment greatly contributed to the successful completion of my work.

Furthermore, I express my heartfelt appreciation to **Guru Nanak Dev Engineering College, Ludhiana**, for the academic foundation and continued support that enabled me to participate in this internship. Lastly, I would like to thank my family, friends, and peers for their encouragement throughout the journey.

List Of Tables

1. Introduction

Background:

Sine waves are fundamental in electronics and signal processing, representing periodic oscillations found in various natural and technological phenomena. They are crucial in understanding concepts such as alternating current (AC), signal modulation, and waveform analysis. The smooth, repetitive oscillations of sine waves are pivotal in the analysis and synthesis of signals in communication systems, audio processing, and control systems.

The emergence of Tiny Machine Learning (Tiny ML) brings the power of machine learning to embedded systems, enabling intelligent data processing on low-power devices. By leveraging Tiny ML, embedded systems can perform tasks like anomaly detection, predictive maintenance, and pattern recognition directly on-device, making them highly efficient and responsive.

Objectives:

The primary objectives of this project are:

1. To generate synthetic sine wave data using mathematical functions and visualize it in real-time
2. To transmit this data to a Digital-to-Analog Converter (DAC) for conversion and display on an oscilloscope.
3. To demonstrate the integration of Tiny ML techniques in processing and analysing the generated data.
4. To showcase the practical applications of Tiny ML in enhancing the capabilities of embedded systems, specifically in signal processing and real-time data analysis.

Scope:

The scope of this project includes:

• Utilizing the Arduino Nano 33 BLE Lite and DAC DFR0972 for data generation and visualization.

• Implementing and demonstrating basic Tiny ML concepts to process sine wave data.

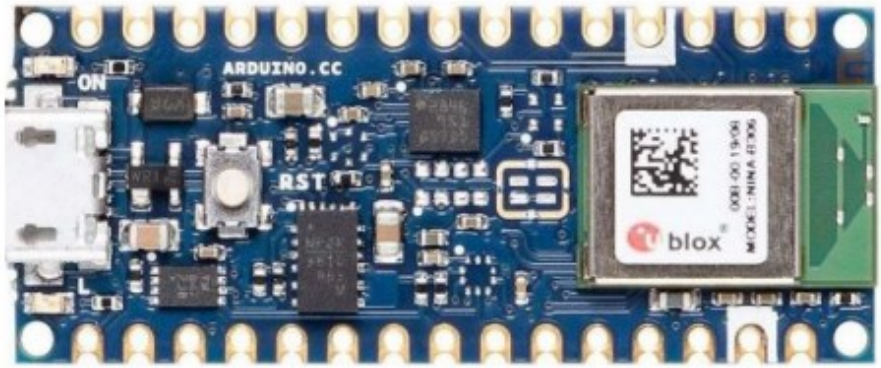
2. Methodology

2.1 Hardware and Software Used:

Hardware Components:

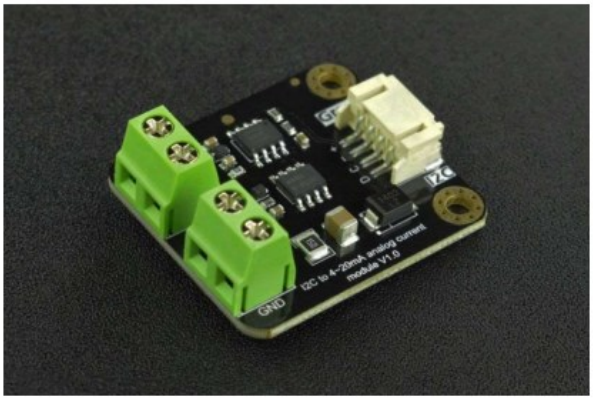
Arduino Nano 33 BLE Lite

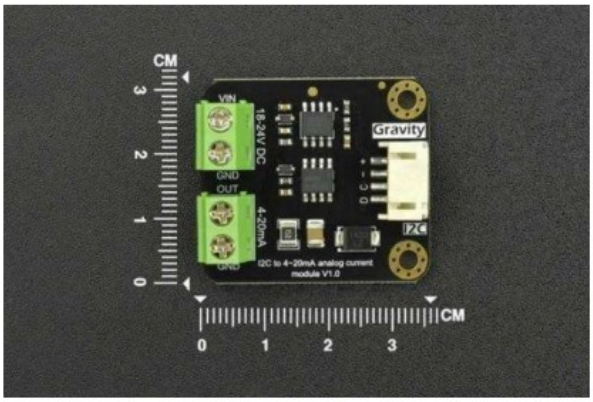
The **Arduino Nano 33 BLE Lite** is a compact and powerful microcontroller board designed for a wide range of applications, including Bluetooth Low Energy (BLE) projects. It offers a combination of performance, connectivity, and flexibility in a small form factor.

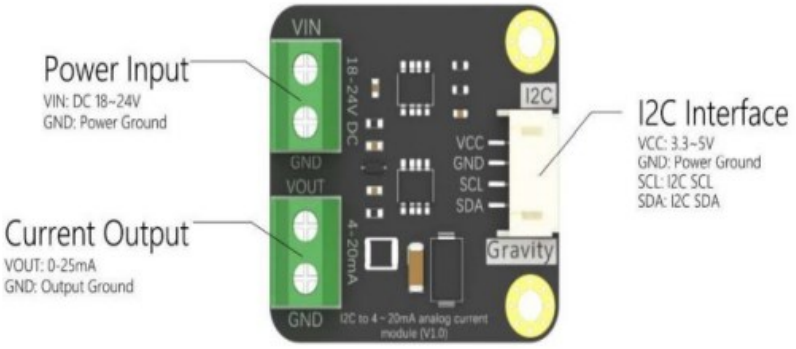


DAC DFR0972:

The **DAC DFR0972** is a Digital-to-Analog Converter (DAC) module designed for precise analog signal output. It is typically used in projects that require the conversion of digital signals from a microcontroller into analog signals, which can then be used to control various analog devices or to interface with analog systems.




CM
3
2
1
0
VIN
18-24V DC
GND
OUT
4-20mA
GND
Gravity
I2C
I2C to 4~20mA analog current module V1.0
CM
0
1
2
3


Power Input
VIN: DC 18~24V
GND: Power Ground
Current Output
VOUT: 0-25mA
GND: Output Ground
VIN
18-24V DC
GND
VOUT
4-20mA
GND
I2C
VCC
GND
SCL
SDA
Gravity
I2C to 4 ~ 20mA analog current module (V1.0)
I2C Interface
VCC: 3.3~5V
GND: Power Ground
SCL: I2C SCL
SDA: I2C SDA

Software Tools:

The **Arduino Integrated Development Environment (IDE)** is a cross-platform application that is essential for programming and controlling Arduino boards. It allows developers to write code, upload it to the microcontroller, and monitor outputs in real time. The IDE supports various programming languages, primarily C and C++, and includes libraries that simplify the interaction with hardware components like sensors, actuators, and communication modules.

The **Arduino IDE** is a crucial tool in this project, as it enables the development and testing of code for the **Arduino Nano 33 BLE Sense** and **DAC module (DFR0972)**, which are used to generate and visualize sine wave data.

Installation of Arduino IDE

1. **Download**:

- Visit the official Arduino website: https://www.arduino.cc/en/software.
- Select the version of the Arduino IDE that corresponds to your operating system (Windows, macOS, Linux).

2. **Windows Installation**:
- Download the **Windows Installer** (.exe file) from the website.
- Run the installer and follow the on-screen instructions to complete the installation.
- Ensure to check the box to install **device drivers** during setup.
- Once installed, launch the **Arduino IDE** from the Start menu.

Setting Up Arduino Nano 33 BLE Sense

1. **Board Manager**:
- Open the **Arduino IDE** and go to **Tools > Board > Board Manager**.
- In the search bar, type "Nano 33 BLE" and install the package for **Arduino DFR0972 boards**.

2. **Library Installation**:
- The **Arduino Tiny Machine Learning Kit** comes with pre-built libraries for machine learning tasks. Go to **Tools > Manage Libraries** and search for **Arduino_TinyML**, then install it.

3. **Connecting the Arduino Board**:
- Connect the **Arduino Nano 33 BLE Sense** to your computer using a USB cable.
- In the Arduino IDE, go to **Tools > Port** and select the port to which the board is connected.
- Select **Arduino Nano 33 BLE** from **Tools > Board**.

4. **Code Uploading**:
- Write your code in the sketch editor or load example code from **File > Examples**.
- Click the **Upload** button to upload the code to the Arduino board.

5. **Serial Monitor**:

- The **Serial Monitor** can be accessed via **Tools > Serial Monitor** to visualize real-time data from the board, including the sine wave data generated in the project.

2.2 Schematic Diagram:

The schematic diagram below illustrates the connections between the **Arduino Nano 33 BLE Sense**, the **DAC module (DFR0972)**, and other peripheral components used in the project. This setup is used to generate sine wave data and transmit it to an oscilloscope for real-time visualization.

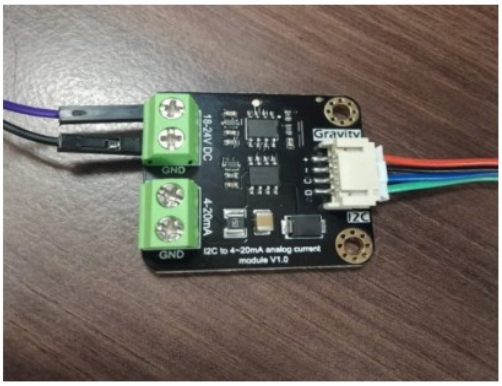
Components:

1. Arduino Nano 33 BLE Lite
2. DAC DFR0972
3. Oscilloscope
4. Connecting Wires

Connections:

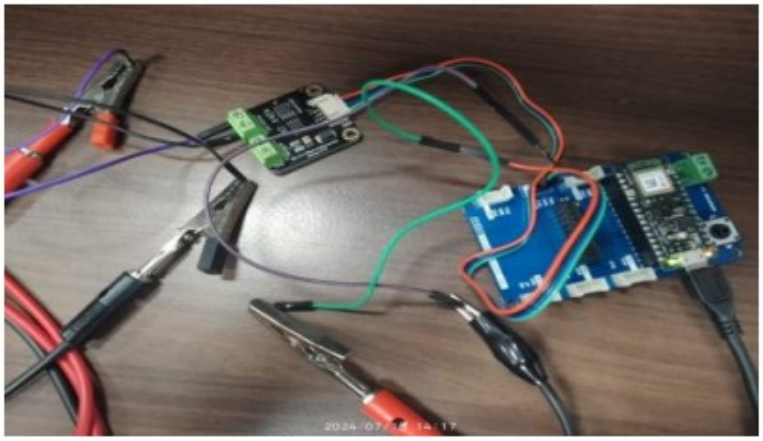
1. **Arduino Nano 33 BLE Lite to DAC DFR0972:**

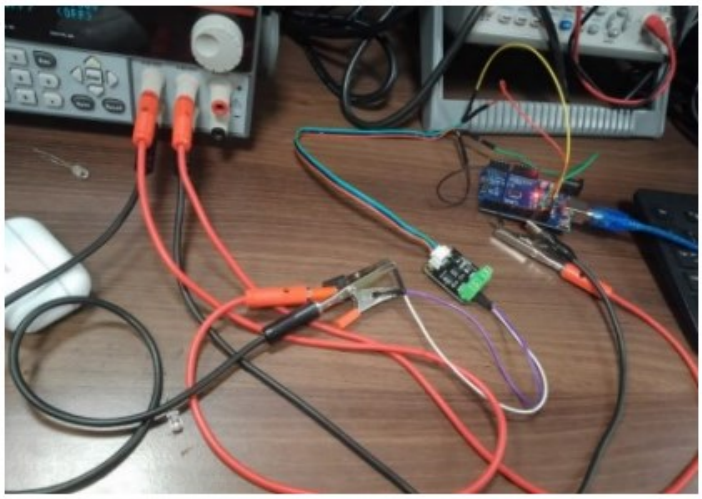
- SDA (Data Line) on Arduino → SDA on DAC
- SCL (Clock Line) on Arduino → SCL on DAC
- GND on Arduino → GND on DAC
- 3.3V or 5V on Arduino → VCC on DAC



2. DAC DFR0972 to Oscilloscope:

- Vout (Analog Output) on DAC → Channel 1 Input on Oscilloscope
- GND on DAC → GND on Oscilloscope.





2.3 Building Our Model:

Here is an code of the sine wave generation might look:(Harvard TinyMLx—Hello world)

```
#include <TensorFlowLite.h>

#include "main_functions.h"
   #include "tensorflow/lite/micro/all_ops_resolver.h"
```

```
#include "constants.h"
#include "model.h"
#include "output_handler.h"
#include "tensorflow/lite/micro/micro_error_reporter.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "tensorflow/lite/version.h"

// Globals, used for compatibility with Arduino-style sketches.
namespace {
tflite::ErrorReporter* error_reporter = nullptr;
const tflite::Model* model = nullptr;
tflite::MicroInterpreter* interpreter = nullptr;
TfLiteTensor* input = nullptr;
TfLiteTensor* output = nullptr;
int inference_count = 0;

constexpr int kTensorArenaSize = 2000;
uint8_t tensor_arena[kTensorArenaSize];
}  // namespace

// The name of this function is important for Arduino compatibility.
void setup() {
      // Set up logging. Google style is to avoid globals or statics because of
  // lifetime uncertainty, but since this has a trivial destructor it's okay.
  // NOLINTNEXTLINE(runtime-global-variables)
  static tflite::MicroErrorReporter micro_error_reporter;
  error_reporter = &micro_error_reporter;

  // Map the model into a usable data structure. This doesn't involve any
  // copying or parsing, it's a very lightweight operation.
  model = tflite::GetModel(g_model);
  if (model->version() != TFLITE_SCHEMA_VERSION) {
    TF_LITE_REPORT_ERROR(error_reporter,
                         "Model provided is schema version %d not equal "
                         "to supported version %d.",
                         model->version(), TFLITE_SCHEMA_VERSION);
    return;
  }

  // This pulls in all the operation implementations we need.
  // NOLINTNEXTLINE(runtime-global-variables)
  static tflite::AllOpsResolver resolver;

  // Build an interpreter to run the model with.
  static tflite::MicroInterpreter static_interpreter(
      model, resolver, tensor_arena, kTensorArenaSize, error_reporter);
  interpreter = &static_interpreter;

  // Allocate memory from the tensor_arena for the model's tensors.
  TfLiteStatus allocate_status = interpreter->AllocateTensors();
```

```
  if (allocate_status != kTfLiteOk) {
    TF_LITE_REPORT_ERROR(error_reporter, "AllocateTensors() failed");
    return;
  }

  // Obtain pointers to the model's input and output tensors.
  input = interpreter->input(0);
  output = interpreter->output(0);

  // Keep track of how many inferences we have performed.
  inference_count = 0;
}

// The name of this function is important for Arduino compatibility.
void loop() {
  // Calculate an x value to feed into the model. We compare the current
  // inference_count to the number of inferences per cycle to determine
  // our position within the range of possible x values the model was
  // trained on, and use this to calculate a value.
  float position = static_cast<float>(inference_count) /
                   static_cast<float>(kInferencesPerCycle);
  float x = position * kXrange;

  // Quantize the input from floating-point to integer
  int8_t x_quantized = x / input->params.scale + input->params.zero_point;
  // Place the quantized input in the model's input tensor
  input->data.int8[0] = x_quantized;

  // Run inference, and report any error
  TfLiteStatus invoke_status = interpreter->Invoke();
  if (invoke_status != kTfLiteOk) {
    TF_LITE_REPORT_ERROR(error_reporter, "Invoke failed on x: %f\n",
                         static_cast<double>(x));
    return;
  }

  // Obtain the quantized output from model's output tensor
  int8_t y_quantized = output->data.int8[0];
  // Dequantize the output from integer to floating-point
  float y = (y_quantized - output->params.zero_point) * output->params.scale;

  // Output the results. A custom HandleOutput function can be implemented
  // for each supported hardware target.
  HandleOutput(error_reporter, x, y);

  // Increment the inference_counter, and reset it if we have reached
  // the total number per cycle
  inference_count += 1;
  if (inference_count >= kInferencesPerCycle) inference_count = 0;
}
```

Visualizing Data Using Serial Plotter

1. **Setup**:
   - Open the Arduino IDE and upload the sketch to the Arduino Nano 33 BLE Lite.
   - Ensure the Serial Plotter is available by navigating to Tools > Serial Plotter in the Arduino IDE.
2. **Visualization**:
   - The Serial Plotter will display a real-time graphical representation of the sine wave data sent from the Arduino.
   - As the data points are transmitted serially, the plot updates dynamically to reflect the oscillatory pattern of the sine wave.

2.4 Data Transfer: From Arduino to DAC Using I2C:

1. **Overview**

The process of transferring data to the DAC (Digital-to-Analog Converter) using I2C and converting it to an analog signal involves several key steps: initializing the I2C communication, sending data to the DAC, and ensuring proper data conversion and output. Here's a detailed breakdown:

2. I2C Communication Setup

I2C Initialization:

- **Library Inclusion**: Ensure that the Wire library, which handles I2C communication, is included in your Arduino sketch. This library provides the necessary functions for communication between the Arduino and I2C devices.

- **Start I2C**: Initialize I2C communication in the setup() function using Wire.begin(). This function configures the Arduino as an I2C master.

3. Data Transmission to the DAC

Addressing the DAC:

- **DAC Address**: Identify the I2C address of the DAC DFR0972. This address is specified in the DAC's datasheet or documentation. For this DAC address is 0x58.

Sending Data:

- **Prepare Data**: Convert the sine wave data to a format suitable for the DAC. Most DACs accept data in a specific resolution (e.g., 12-bit), so scale the sine wave values accordingly.

- **Write Data**: Use the Wire.beginTransmission() function to start communication with the DAC, followed by Wire.write() to send the data, and Wire.endTransmission() to end the communication.

Here's a complete code for integrating data generation, transmission, and DAC output:

```
#include <Wire.h>

const int numPoints = 100;

const float pi = 3.14159;

float sineWave[numPoints];

const int dacAddress = 0x58; // DAC I2C address

void setup() {

  Wire.begin();

  generateSineWave();
```

```
}


void loop() {
  for (int i = 0; i < numPoints; i++) {
    uint16_t dacValue = (sineWave[i] + 1.0) * 2047; // Convert to 12-bit value
    sendDataToDAC(dacValue);
    delay(50); // Delay to simulate waveform frequency
  }
}


void generateSineWave() {
  for (int i = 0; i < numPoints; i++) {
    float angle = (2 * pi / numPoints) * i;
    sineWave[i] = sin(angle);
  }
}


void sendDataToDAC(uint16_t value) {
```

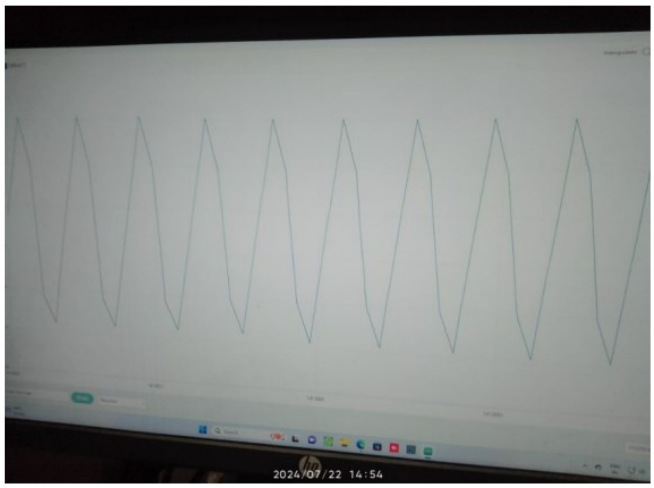
```
  Wire.beginTransmission(dacAddress);

  Wire.write((value >> 8) & 0xFF); // High byte

  Wire.write(value & 0xFF); // Low byte

  Wire.endTransmission();

}
```

3 . Results and Discussions

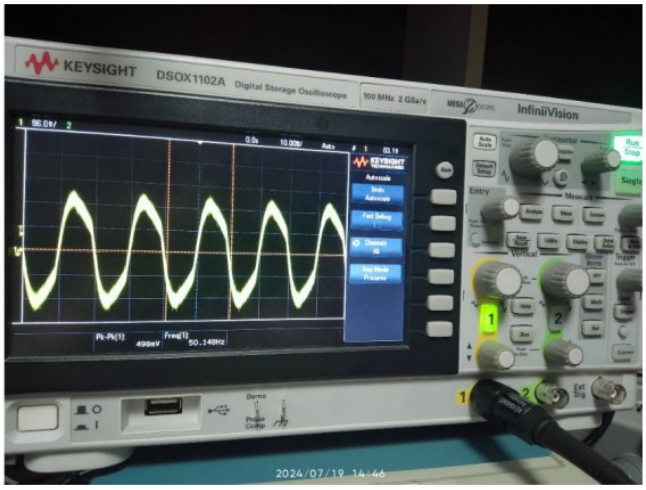
Generated Sine Waves:

Data Presentation:

- **Serial Plotter**: The sine wave data generated by the Arduino Nano 33 BLE Lite was visualized using the Serial Plotter. The plot displayed a smooth, periodic oscillation, characteristic of a sine wave. Below is an example screenshot of the sine wave as seen on the Serial Plotter:



- **Oscilloscope:** The same sine wave data was transmitted to the DAC DFR0972 and visualized on an oscilloscope. The oscilloscope showed a clean, continuous sine wave signal, demonstrating the DAC's capability to accurately convert digital data into analog form. Below is an example image of the sine wave displayed on the oscilloscope:



4 .Conclusion

Summary of Findings

The project successfully demonstrated the generation and visualization of sine wave data using the Arduino Nano 33 BLE Lite and the DAC DFR0972. Key findings include:

1. **Sine Wave Generation**: The Arduino was able to generate smooth and accurate sine wave data using the mathematical sine function y=sin(x). This data was transmitted to the DAC, which converted it into an analog signal.

2. **Visualization**: The generated sine wave was effectively visualized using the Arduino Serial Plotter and an oscilloscope. The Serial Plotter displayed the sine wave in real-time, while the oscilloscope confirmed the analog signal's accuracy and fidelity.

3. **Data Accuracy**: The analysis showed that the sine wave data closely matched the theoretical expectations. The amplitude and frequency were consistent with the intended values, indicating that both the data generation and conversion processes were accurate.

Overall, the project validated the integration of TinyML concepts with embedded systems for real-time signal processing and demonstrated the practical application of microcontrollers and DACs in generating analog signals.

5 .References

1. **Book:**

- Warden, P., & Situnayake, D. (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media. Download

2. **Datasheets:**

- Arduino Nano 33 BLE Lite, Download
- DAC DFR0972: DFRobot. Download

3. **Tools and Software:**

- Arduino IDE
- Libraries

This list includes the primary references used in the preparation of the project, covering essential books, datasheets, and tools relevant to the work on sine wave generation and TinyML applications.